Unit 14 Installing Aras Innovator Applications

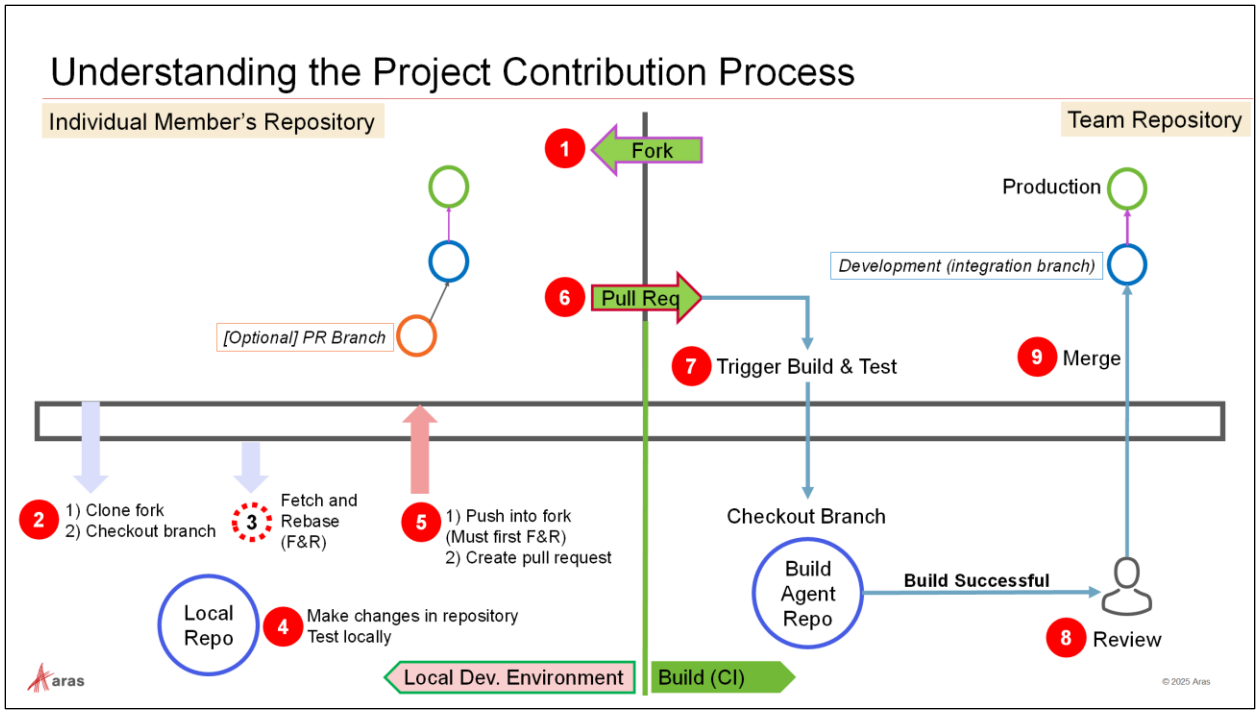
Overview

In this unit, you will learn how to **install and manage Aras Innovator applications** within the **Aras DevOps framework**.
This process is often required after **Aras Global Cloud Services (GCS)** delivers the initial customer project to establish a **baseline** that includes specific Aras Innovator applications.

You will also learn how to **prepare both the Standard Development Environment (SDE)** and the **Local Development Environment (LDE)** to support new installations.
After completing the setup and installation, you will **push applications from the LDE to the SDE** and finally **deploy the applications** in a new **System Integration Testing (SIT)** instance.

Objectives

- **Prepare the SDE** for new development work
- **Identify and verify environment prerequisites** for the LDE
- **Set up the LDE** to support new Aras Innovator applications
- **Install and configure Aras Innovator applications** in the LDE
- **Validate installations** to ensure successful deployment in the LDE
- **Share and synchronize applications** within the SDE
- **Validate Aras Innovator applications** in the SDE environment
- **Finalize the project** by establishing a new baseline for future development



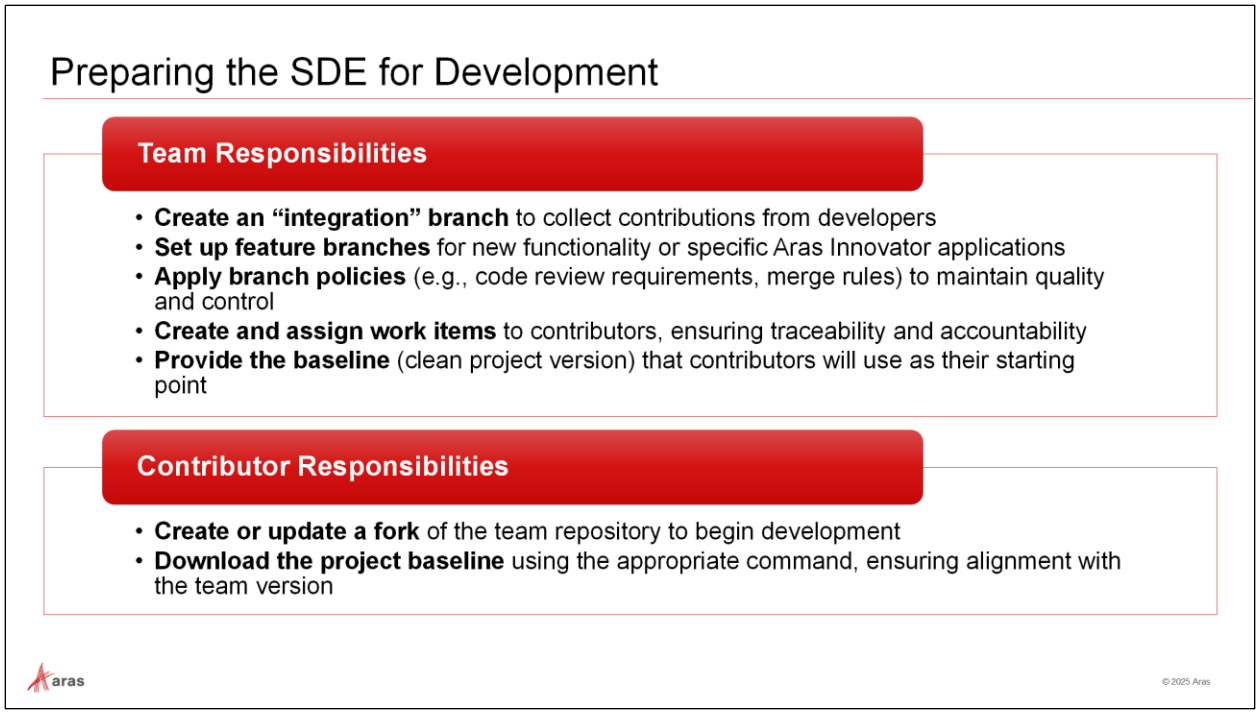
Understanding the Project Contribution Process

Purpose

The goal of this process is to ensure that **code contributions from multiple contributors** are properly reviewed, tested, and integrated into the **team repository** using Git best practices.
It supports **version control**, **collaboration**, and **continuous integration (CI)** in Aras DevOps.

Process Overview

Step	Description
1	Create a Fork (copy of the shared team repository) in SDE (Aras Azure DevOps).
2	Clone your Fork to your local environment and check out the branch you wish to work on.
3	This is a step that is executed regularly (at least once a day before starting your work). You need to add a remote reference to the team repo, so that you can fetch and rebase your current work on it. This process reduces (or even eliminates) the chance that your eventual PR is rejected because of conflicts. You should also do the same with your Fork, particularly if others have access to it and can update it through a PR.
4	Work in LDE – execute **RunIntegrationTests**, **ContinuousIntegration**, and other scripts.
5	Commit your changes locally, execute a Fetch and Rebase operation, and then push to Fork.
6	Create your pull request in SDE to share your local changes.
7	The creation of the pull request, with the updates to your fork source branch, that you want to share with the team target branch, will trigger a **ContinuousIntegration** pipeline execution, based on branch policy.
8	When the **CI** pipeline has successfully built your artifact, it will run tests and report a success status (green) or failure status (red). Based on that status, reviewers may approve your PR. They may also reject it even with a green build, if some other project policies are not fulfilled.
9	When the PR is approved and merged, the target branch would typically have a policy to run the **ContinuousIntegration** pipeline again. This is to make sure that all PRs are still in sync. The goal is to ensure that the common repo target branch is always buildable.



Preparing the SDE for Development

Before new development work begins, both the **SDE** and **LDE** must be properly prepared.
This preparation ensures that contributors can work efficiently, maintain consistency, and align with project governance standards.

Team Responsibilities

The following steps are recommended for the **project team** when preparing the SDE:

- **Verify the baseline**: Confirm that the initial baseline delivered by **Aras Global Cloud Services (GCS)** is available in Azure DevOps under
  Artifacts > DevOps-Baselines or **Artifacts > SaaS-Baselines**.
- Create an "integration" branch in the Team repository.
  - This branch receives all contributions from developers.
  - It should be created from the existing **development branch** (provided by Aras and synchronized with the latest production deployment).
- **Create feature branches**, as needed, to separate the installation of specific **Aras Innovator applications** or testing work.
- **Apply branch policies** to enforce quality control and governance for both integration and feature branches.
- The **team lead or project manager** creates and assigns **work items** to contributors for clear task ownership and traceability.

Contributor Responsibilities

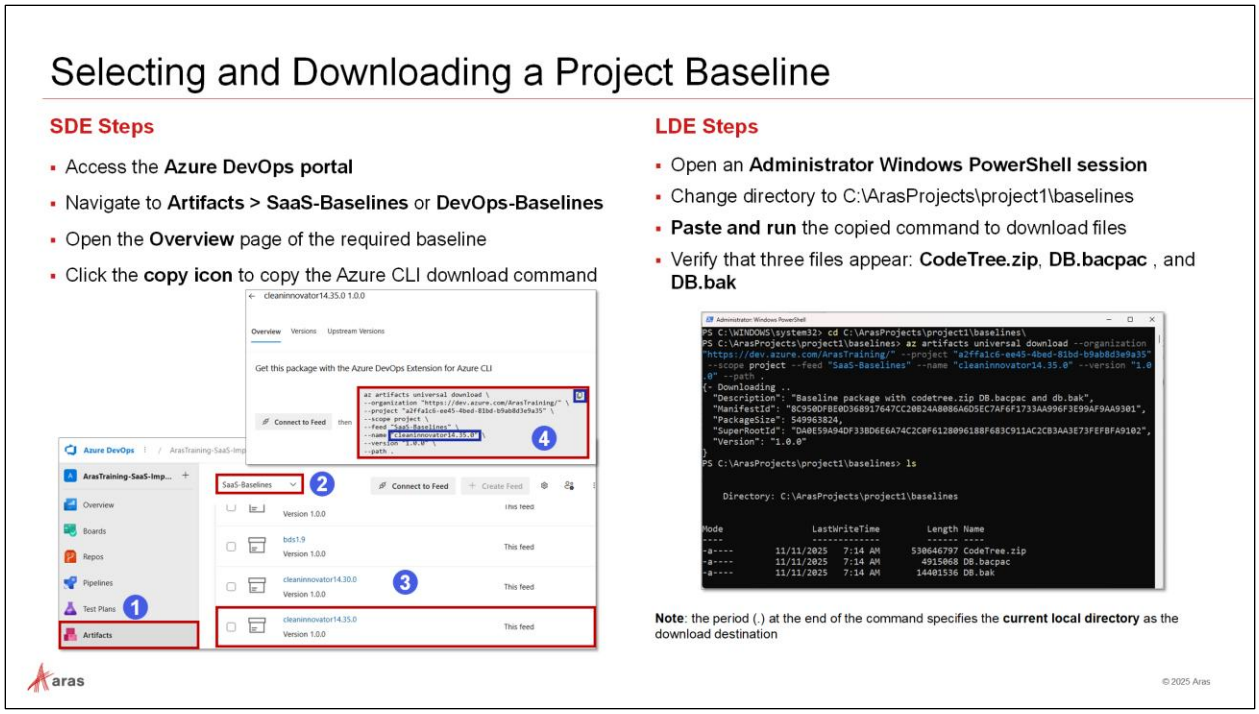
The following steps are recommended for **contributors** when setting up their environment in the SDE:

- **Create a new fork** of the Team repository (including all branches). Alternatively, reuse an existing fork by creating a new branch that matches the name assigned by the team lead.
- Ensure the new branch originates from the same **source branch** as specified by the team.

- **Download the assigned project baseline** from Azure DevOps Artifacts.

Notes:

- A properly prepared SDE ensures **consistency, stability, and smooth collaboration** across all team members before development begins
- Branch policies and baselines maintain synchronization between individual work and the shared team environment



Selecting and Downloading a Project Baseline

The goal of this process is to **retrieve the correct project baseline** from the **SDE** in Azure DevOps and download it to your **LDE**. This ensures that all contributors work from the same validated starting point.

Selecting the Baseline

A baseline establishes the starting point for the build tools. The Git delta, which contains the desired Aras Innovator applications, will be applied from that point forward allowing a new build to complete.

Lab 1: Select the Baseline

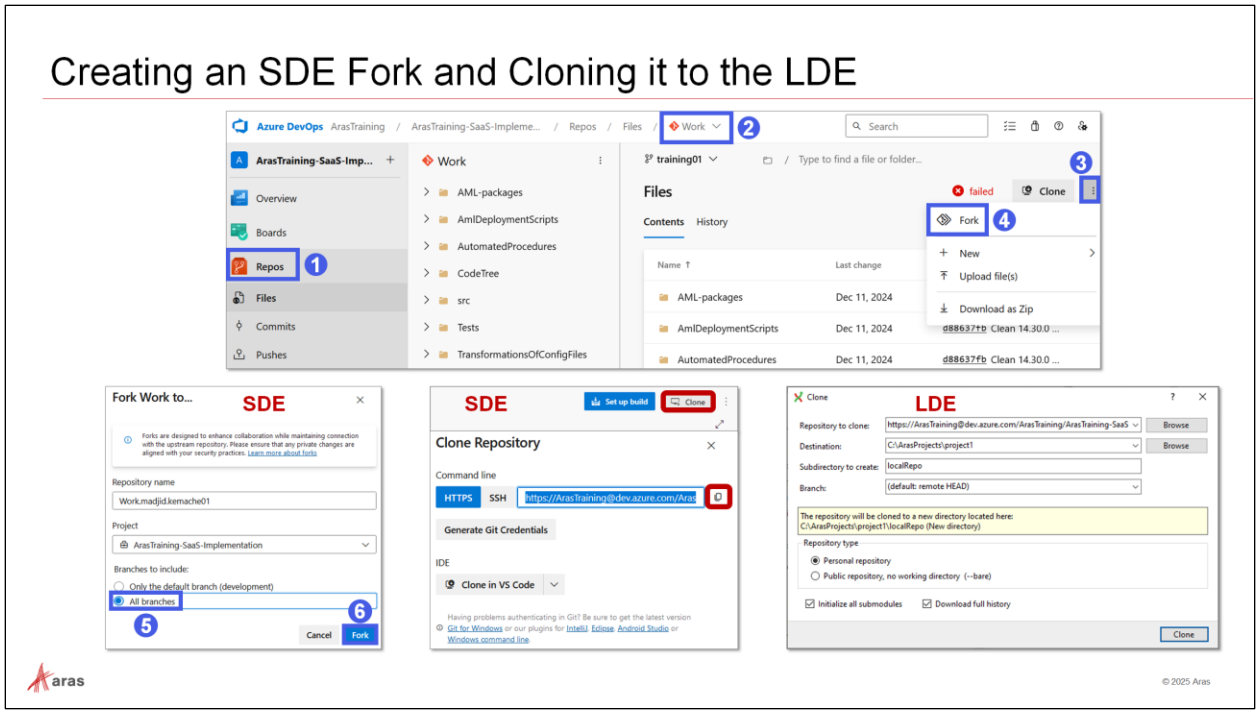
1. In a web browser, connect to the SDE, i.e., https://dev.azure.com/ArasTraining.
2. Go to **Artifacts** to find the baseline feed (**SaaS-Baselines** or **DevOps-Baselines**)
3. Click the link for your project baseline (e.g., **cleaninnovator14.35.0**)
4. On the **Overview** page, click the **copy icon** to copy the Azure CLI download command

Downloading the Baseline

Normally, the latest baseline, which represents the current production, is available as the starting point for Aras Innovator instance customizations in the LDE.

Lab 2: Download the Baseline

1. Open an **Administrator Windows PowerShell** session
2. **Navigate** to the target directory, e.g.
   cd C:\ArasProjects\project0\baselines
3. **Paste and run** the copied command from Azure DevOps
4. Verify the command completes successfully and that the following files appear:
   - **CodeTree.zip**
   - **DB.bacpac**
   - **DB.bak**



Creating an SDE Fork and Cloning it to the LDE

A **fork** is a **personal repository** duplicated from the team repository. It resides in the **SDE (remote)** and serves as your **personal workspace** for development.

The key advantage of this approach is **protection of the main team repository**—ensuring that production code remains stable while contributors develop and test changes independently.
In the standard Aras DevOps review workflow, **only authorized reviewers** can merge proposed changes into the team repository.

Lab 1: Create a Personal Fork in the SDE

1. In the left-hand menu of your Azure DevOps project, click **Repos**.
2. In the breadcrumb navigation at the top of the page, select the **Work** team repository from the drop-down.
3. In the upper-right corner, click the **More actions (⋯)** icon.
4. From the menu, select **Fork**.
5. In the **Fork Work To...** dialog box:
   Update the **Repository name** following this format:
   Work.AxxTrngNN *(replace "xx" and "NN" with the values provided by your instructor)*
6. Under **Branches to include**, select **All branches**.
7. Click **Fork**.
8. To verify that your fork was created successfully and includes all branches:
   a. In the left-hand menu, click **Repos → Branches**.
   b. On the **Branches** page, select the **All** tab.
   c. Confirm that the **class-specific branch** (as assigned by your instructor) is listed.

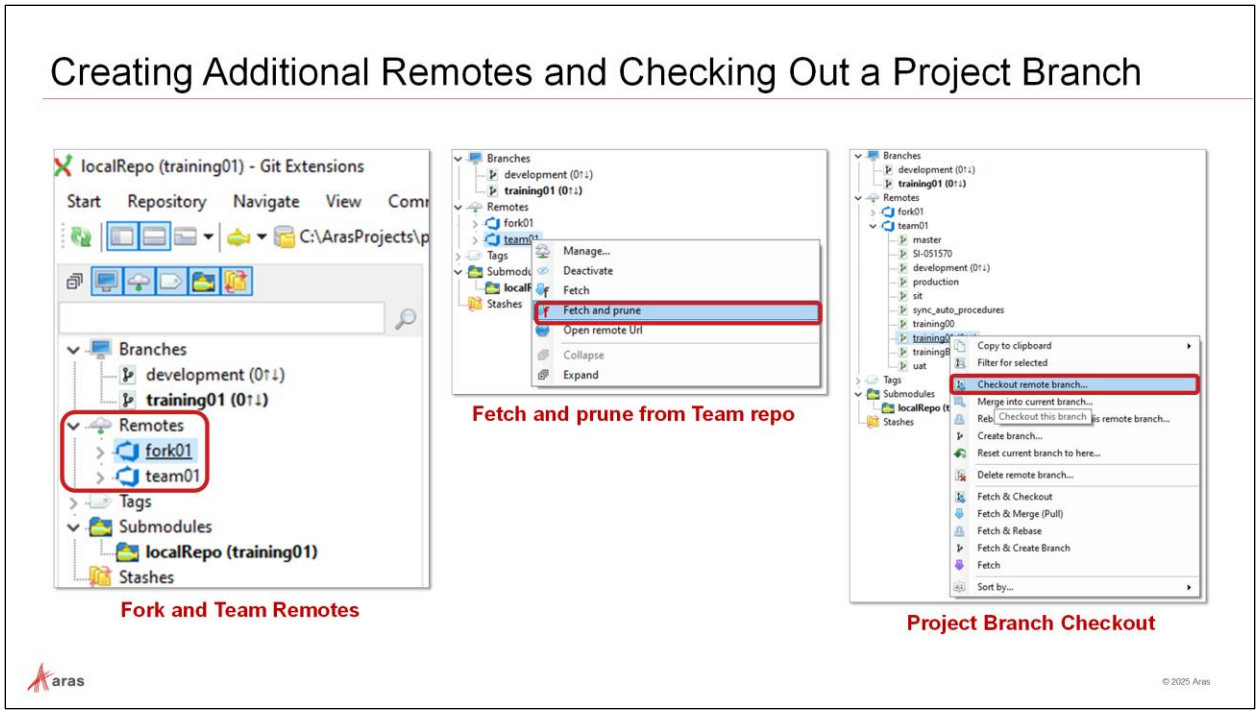
Result: The fork now isolates production code from your development work.
All modifications—whether bug fixes or Aras Innovator customizations—will now occur in your **Local Development Environment (LDE)**.

Lab 2: Clone the SDE Fork into LDE Using Git Extensions

1. In Azure DevOps, click **Repos** in the left-hand menu
2. In the upper-right corner, click the **Clone** button
3. In the pop-up window, click the **Copy clone URL** icon
4. In **File Explorer**, navigate to C:\ArasProjects\project0
5. Right-click the project0 subfolder and select **GitExt Clone…**
6. In the **Clone Repository** window, complete the following fields:
   a. **Repository to clone:** Paste the copied URL from your fork
   b. **Destination:** C:\ArasProjects\project0
   c. **Subdirectory to create:** localRepo
7. Leave the remaining fields set to their default values
8. Click **Clone**, then click **OK** when complete
9. Verify that the contents of the localRepo folder match those of your SDE fork

Notes:

- This step establishes the **developer's LDE workspace**, fully synchronized with the SDE fork.
- It ensures safe, traceable development and aligns all contributors with a consistent, version-controlled baseline.



Creating Additional Remotes and Checking Out a Project Branch

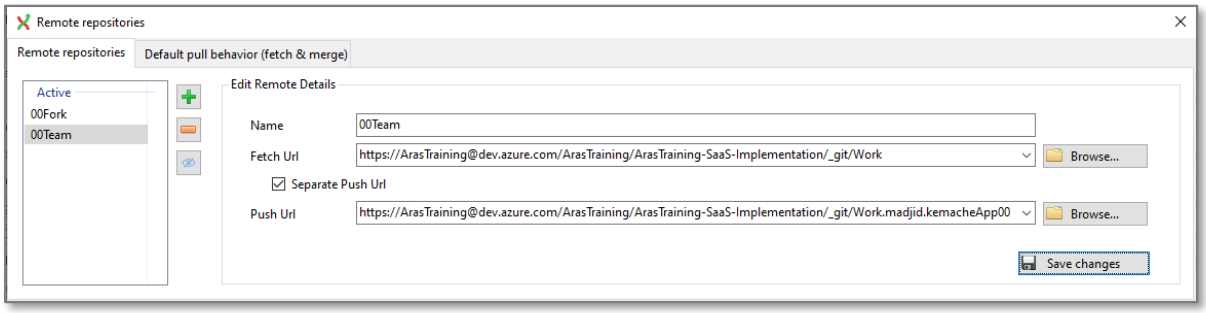
In Git, **remotes** provide a way to track and synchronize changes across multiple repositories.
When you clone your Azure DevOps fork, a **remote reference to your fork** is automatically created.

To stay synchronized with the team's shared repository, **Aras DevOps recommends adding a second remote reference** that points to the **Team repository**.
This allows you to fetch updates and ensure your local work remains aligned with the latest team progress.

Lab 1: Create Additional Remotes

1. In **Windows File Explorer**, navigate to:
   C:\ArasProjects\project0\localRepo
2. Right-click and select **GitExt Open repository** from the context menu.
3. In Git Extensions, right-click the **Remotes** node and select **Manage…**.
4. In the **Remote repositories** window, click the **green plus button (+)** to add a new remote.
5. In the **Create New Remote** window, check the **Separate Push URL** box
6. Complete the following fields:
   a. **Name:** *xxTeam* (replace *xx* with the value provided by your instructor)
   b. **Fetch URL**:
   https://ArasTraining@dev.azure.com/ArasTraining/ArasTraining-SaaS-Implementation/_git/Work
   c. **Push URL:**
   https://ArasTraining@dev.azure.com/ArasTraining/ArasTraining-SaaS-Implementation/_git/Work.AxxTrngNN
7. Click **Save changes** to create the new remote reference.
8. When prompted in the **New Remote** dialog, click **Yes**.
9. Select the **origin** remote in the list.

10. In the **Edit Remote Details** box, rename it to *xxFork* (replace *xx* as provided by the instructor).
11. Click **Save changes**.
12. Verify in the **Active Remotes** list that both are visible:
    a. **xxFork** → your personal fork
    b. **xxTeam** → the shared team repository



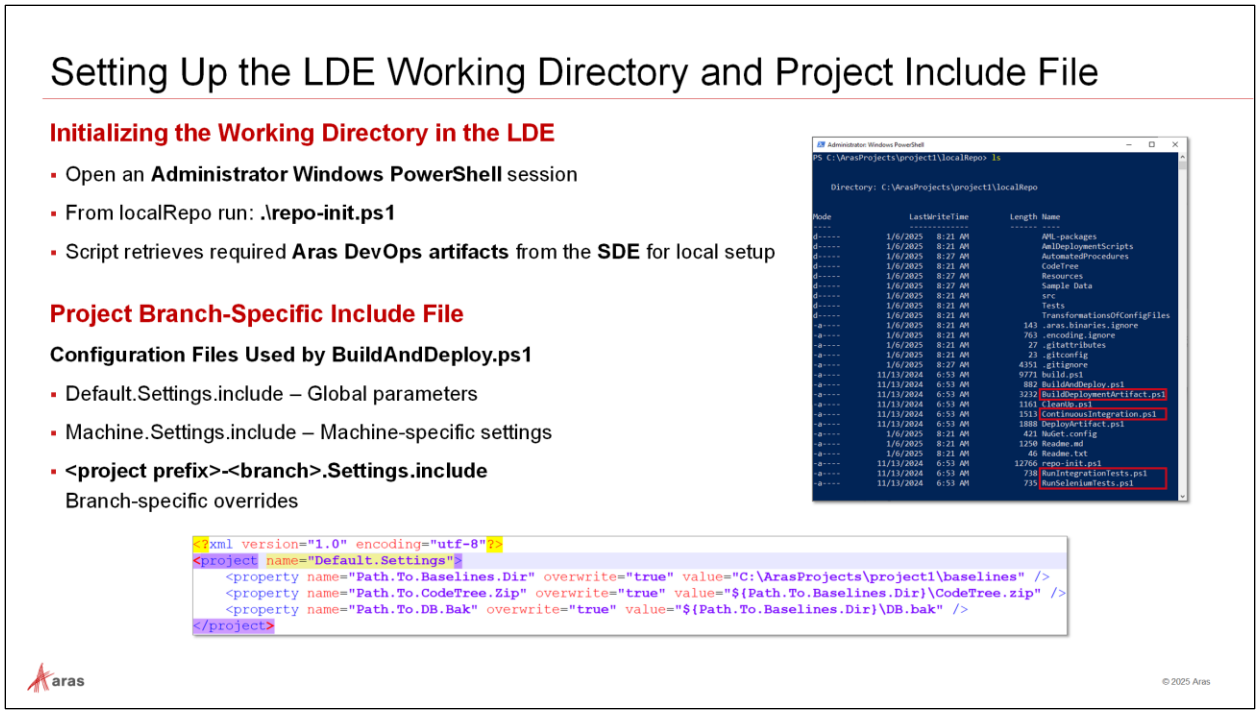
13. Close the **Remote repositories** window.

Result: You now have two remotes configured — one pointing to your fork, and one to the team repository — allowing synchronized collaboration while keeping your work isolated.

Lab 2: Check Out the Project Branch

By default, the cloned repository is checked out to the **development** branch. In this training, you will check out a class-specific branch (AxxTrngNN), previously created by the instructor.

1. In **Windows File Explorer**, navigate to:
   C:\ArasProjects\project0\localRepo
2. Right-click and select **GitExt Open repository**.
3. In the left panel, under **Remotes**, right-click the **xxTeam** repository.
4. Choose **Fetch and Prune**.
5. When prompted (*Prune remote branches from xxTeam?*), click **Yes**.
6. In the **Process** dialog, click **OK**.
7. Expand the **xxTeam** remote and locate the assigned branch (AxxTrngNN).
8. Right-click the branch and select **Checkout remote branch…**.
9. In the **Checkout Branch** window, accept the default options and click **Checkout**.
10. In the **Process** dialog, click **OK**.
11. Confirm the new branch (AxxTrngNN) appears under **Branches** in bold — indicating it is checked out.
12. In **Windows File Explorer**, navigate to:
    C:\ArasProjects\project0\localRepo
13. Verify that the repository's **starting files** are now loaded in your local environment.



Setting Up the LDE Working Directory and Project Include File

In the labs below, you will initialize your **LDE** working directory and prepare the **project branch-specific include file** required by the Aras DevOps build process.
This step ensures your local environment retrieves all necessary **Aras DevOps artifacts** from the **SDE** and that your configuration files are ready for the build and deployment pipeline.

Lab 1: Initialize the Working Directory

1. Open an administrator Windows PowerShell session
2. Change directory to **C:\ArasProjects\project0\localRepo**
3. Run the following script: *.\repo-init.ps1*

Note: This script prepares the local workspace and downloads the required DevOps resources from the SDE, enabling you to run pipelines and build Aras Innovator instances locally.

Configuration Files Used During Build

The **BuildAndDeploy.ps1** pipeline relies on three configuration files, processed in the following order:

1. **Default.Settings.include**
   C:\ArasProjects\project0\localRepo\AutomatedProcedures\Default.Settings.include
   → Contains global parameters and baseline configuration.
2. **Machine.Settings.include**
   In C:_machine_specific_includes
   → Defines machine-specific variables and local paths.
3. **<project prefix>-<branch>.Settings.include**
   In C:_machine_specific_includes
   → Defines branch-specific overrides for project or training environments.

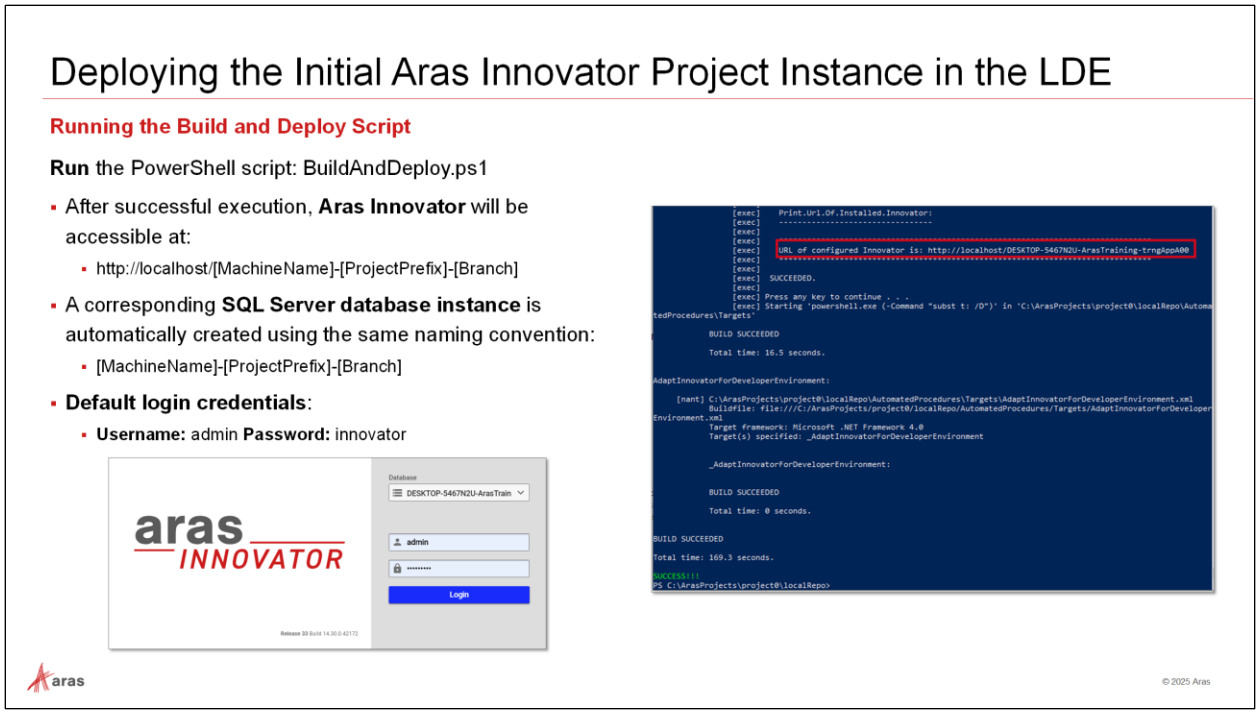
In the lab below, you will create a **branch-specific include file** that defines configuration settings unique to your project branch.
This file provides the **final layer of customization** used by the **BuildAndDeploy.ps1** pipeline during the deployment of your local Aras Innovator instance.

Lab 2: Create the Project Branch-Specific Include File

1. Go to **C:_machine_specific_includes**.
2. Create a text document named
   ArasTraining-AxxTrngNN.Settings.include
   (Replace AxxTrngNN with the branch identifier provided by your instructor.)
3. Copy the contents of the **Machine.Settings.include** file to the newly created file.
4. **Retain and update** only the key properties for your current project and branch:
   a. **Path.To.Baselines.Dir** – Path to the *baselines* folder currently in use.
   b. **Path.To.CodeTree.Zip** – Path to the *CodeTree.zip* archive used by the **BuildAndDeploy.ps1** pipeline.
   c. **Path.To.DB.Bak** – Path to the *database backup (DB.bak)* file used by the **BuildAndDeploy.ps1** pipeline.

Notes:

- You now have a **branch-specific include file** that overrides machine-level configuration values, ensuring the **BuildAndDeploy pipeline** references the correct project, branch, and environment files for your local deployment.
- Each include file corresponds to a **unique project branch**.
  This approach enables parallel testing and customization without affecting shared configurations or other developers' environments.



Deploying the Initial Aras Innovator Project Instance in the LDE

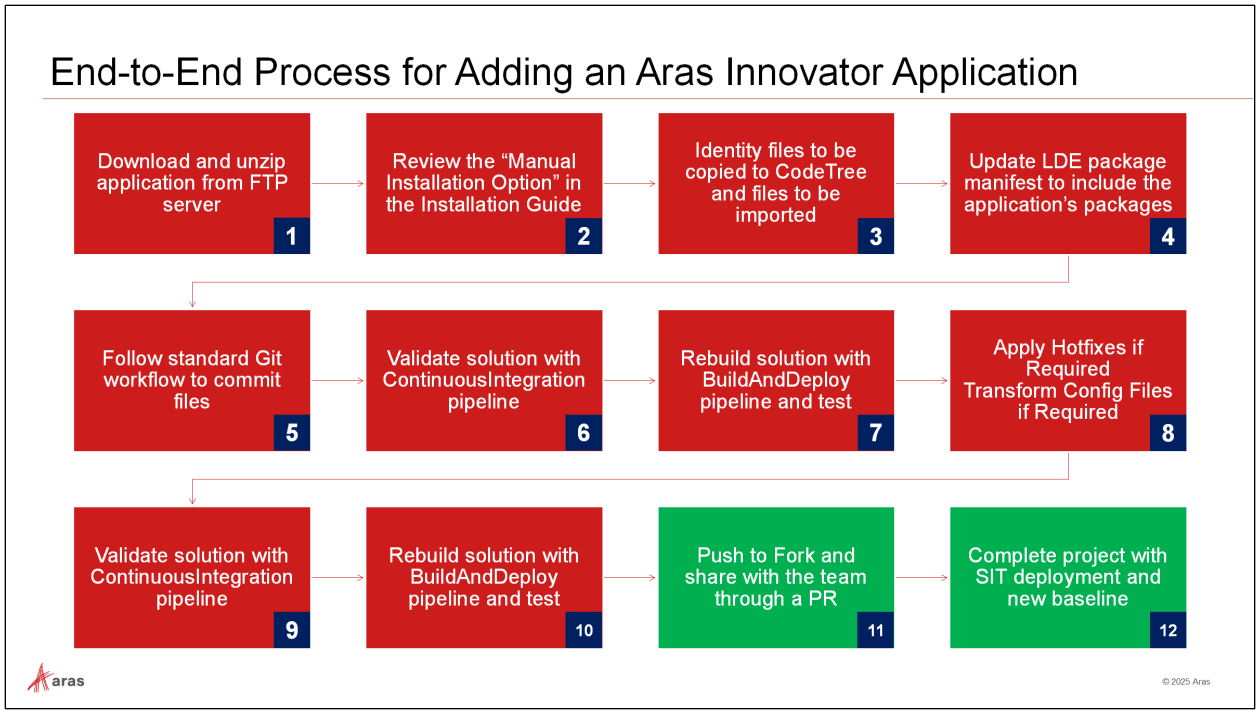
The **LDE repository** needs to be set up with an Aras Innovator project instance, along with automation and testing scripts.

Verify that all required values are correctly configured in both:

- **Machine.Settings.include**
- **ArasTraining-AxxTrngNN.Settings.include**

Lab: Deploy the Initial Aras Innovator Project Instance in the LDE

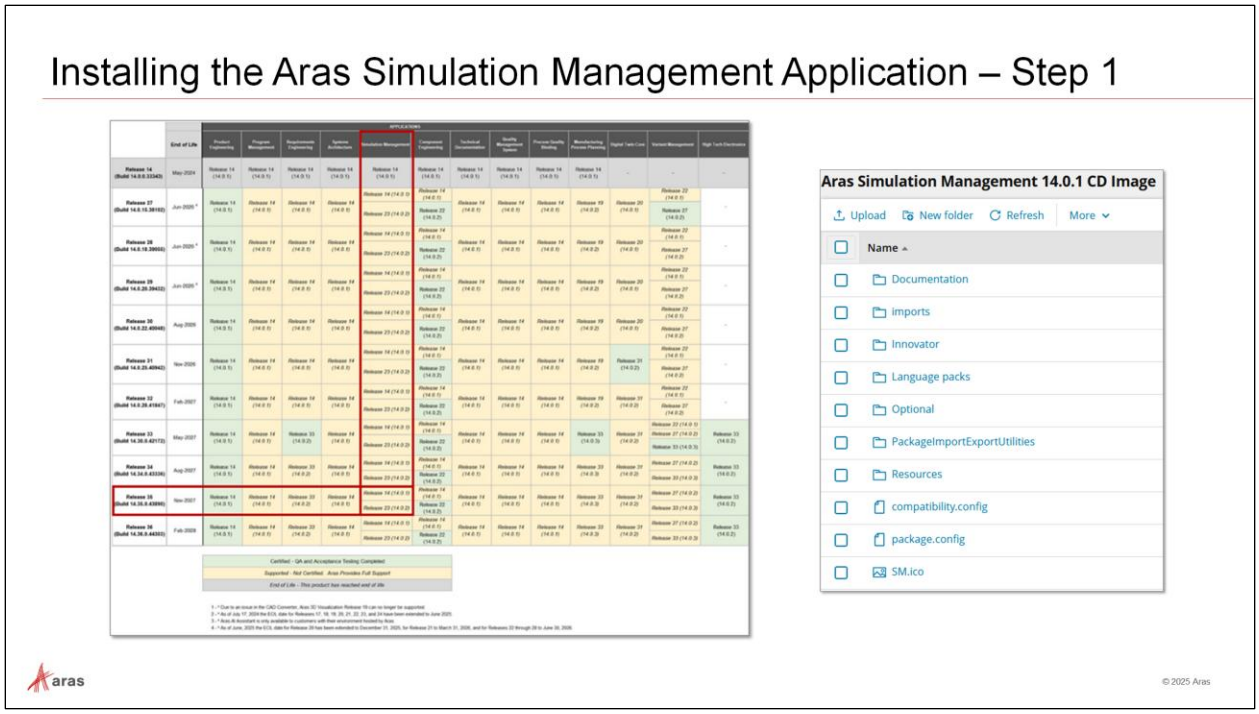
1. Open an **Administrator Windows PowerShell** session
2. In Windows file Explorer, navigate to: **C:\ArasProjects\project0\localRepo**
3. Run the deployment script: **.\BuildAndDeploy.ps1**
4. If any errors occur, review and correct the settings files, then rerun the script in a new admin Windows PowerShell session.
5. When the script completes successfully, **copy the URL** of the new LDE instance displayed in the Windows PowerShell output.
6. Verify deployment by logging in to **Aras Innovator** using the credentials:
   Username: **admin** Password: **innovator**



End-to-End Process for Adding an Aras Innovator Application

The process described above (and summarized in the steps below) outlines the general workflow for installing most Aras Innovator applications. Some applications may require additional or fewer steps, as documented in the official **Aras Innovator Installation Guides** available here: https://docs.aras.com.

Step	Description
1	Download the appropriate version of the application that matches your Aras Innovator release.
2	Review the *Manual Installation Option* section in the **Installation Guide**, and the **SaaS Installation Guide** (if available).
3	Identify the files that must be copied to the **CodeTree** folder in the LDE.
4	Update the **LDE package manifest** to include the new application packages.
5	Follow the standard **Git workflow** to stage and commit the updated and new files in the LDE repo.
6	Validate the solution by running the **ContinuousIntegration** pipeline.
7	Rebuild the solution using the **BuildAndDeploy** pipeline, confirm successful installation, and test it in the Aras Innovator client.
8	Apply available **hotfixes** and perform **configuration transformations**, if required, following the application documentation.
9	Revalidate the solution by rerunning the **ContinuousIntegration** pipeline.
10	Rebuild and redeploy the solution using the **BuildAndDeploy** pipeline and perform verification tests in the client.
11	Push the updated solution to your **Fork** and create and process a new **Pull Request (PR)**.
12	Finalize the project by deploying and testing an **SIT instance** and generating a **new baseline** for future development.



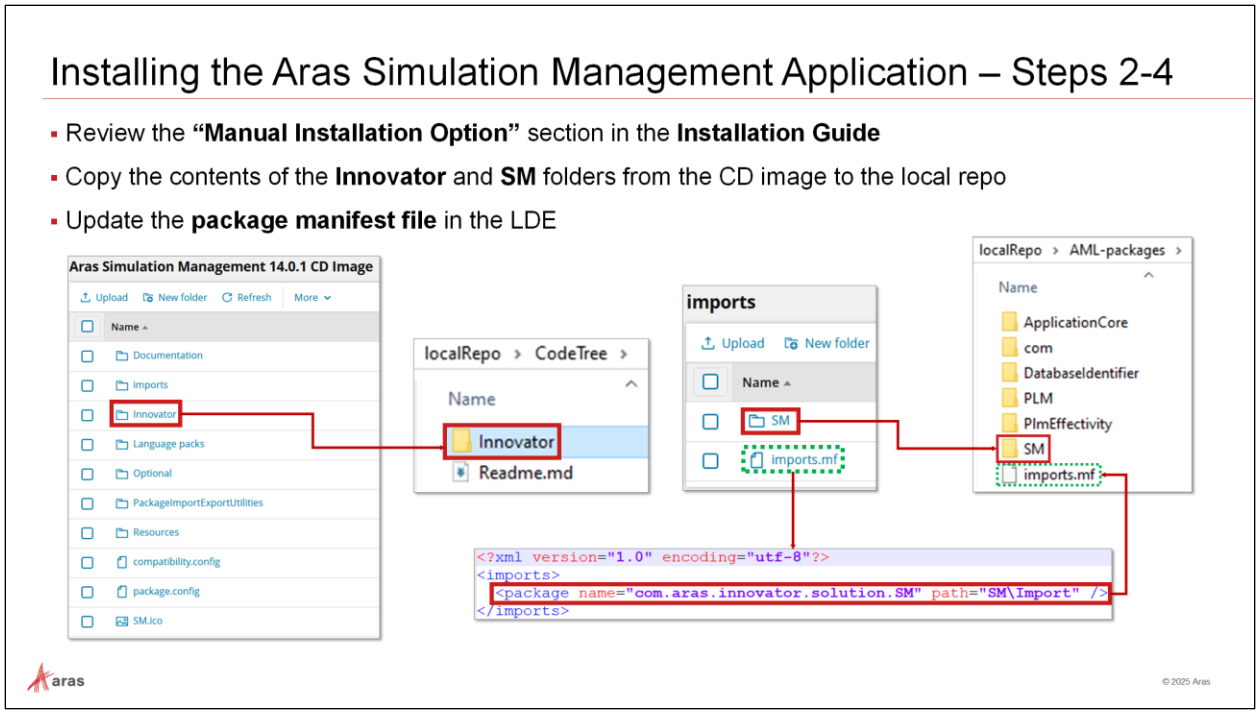
Installing the Aras Simulation Management Application – Step 1

In this lab, you will download the appropriate version of the **Aras Simulation Management (SM)** application from the Aras Files.com site.
This version must match the **Aras Innovator release** currently installed in your **Local Development Environment (LDE).**

Lab: Download the Aras Simulation Management Application Files

1. Determine the required version of **Aras Simulation Management** by comparing your **LDE Aras Innovator version** with the corresponding entry in the **Aras Support Matrix** : https://www.aras.com/community/subscriber-portal/p/support-matrix
2. Download the **Aras Simulation Management CD image (.zip)** from the **Aras Files.com site**.
3. Extract the contents of the downloaded archive to the following folder: C:\ArasProjects\project0\localExports

Note:
Verify the **Aras Innovator platform version** before downloading. Using a mismatched application package will cause validation or build errors in later steps.



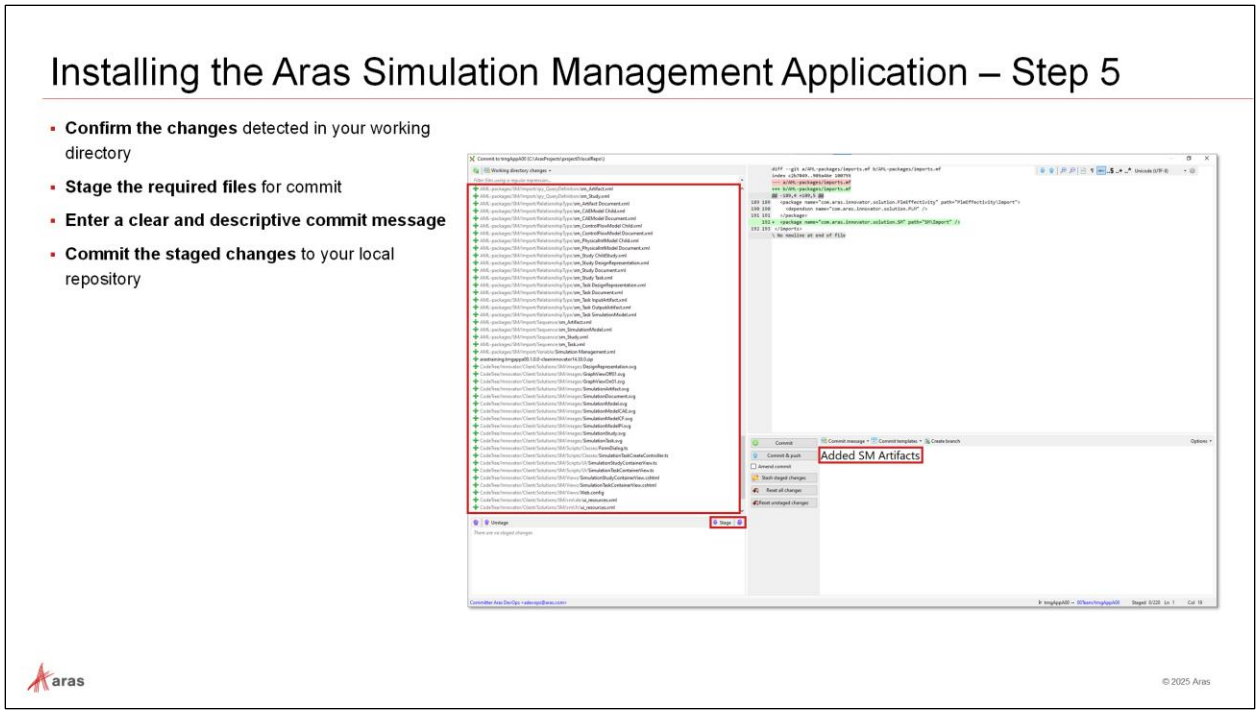
Installing the Aras Simulation Management Application – Steps 2-4

In this lab, you will prepare the Simulation Management (SM) application for installation by reviewing the installation guide, copying necessary files, and updating the package manifest.

Lab: Prepare and Configure the Simulation Management Application

1. Open the extracted **Aras Simulation Management CD Image** folder.
2. Review the **"Manual Installation Option"** section in the **Installation Guide**.
   a. This section identifies the files that must be manually copied to the **CodeTree**.
   b. Review the **SaaS Installation Guide** if available
3. Copy the contents of the **Innovator** folder from the CD image to the following location in the LDE repository:
   ..localRepo\CodeTree\Innovator
4. Add the extracted **SM** package to the following location in the LDE repository:
   ..\localRepo\AML-packages
5. Update the **package manifest file** in your LDE:
   a. Add the **Simulation Management (SM)** package reference in:
      ..\localRepo\imports\imports.mf
   b. Example entry:
      <package name="com.aras.innovator.solution.SM" path="SM\Import" />

Note:
The imports.mf file defines the list of applications to be imported during deployment. Omitting this entry will cause the **BuildAndDeploy.ps1** script to skip the Simulation Management package.



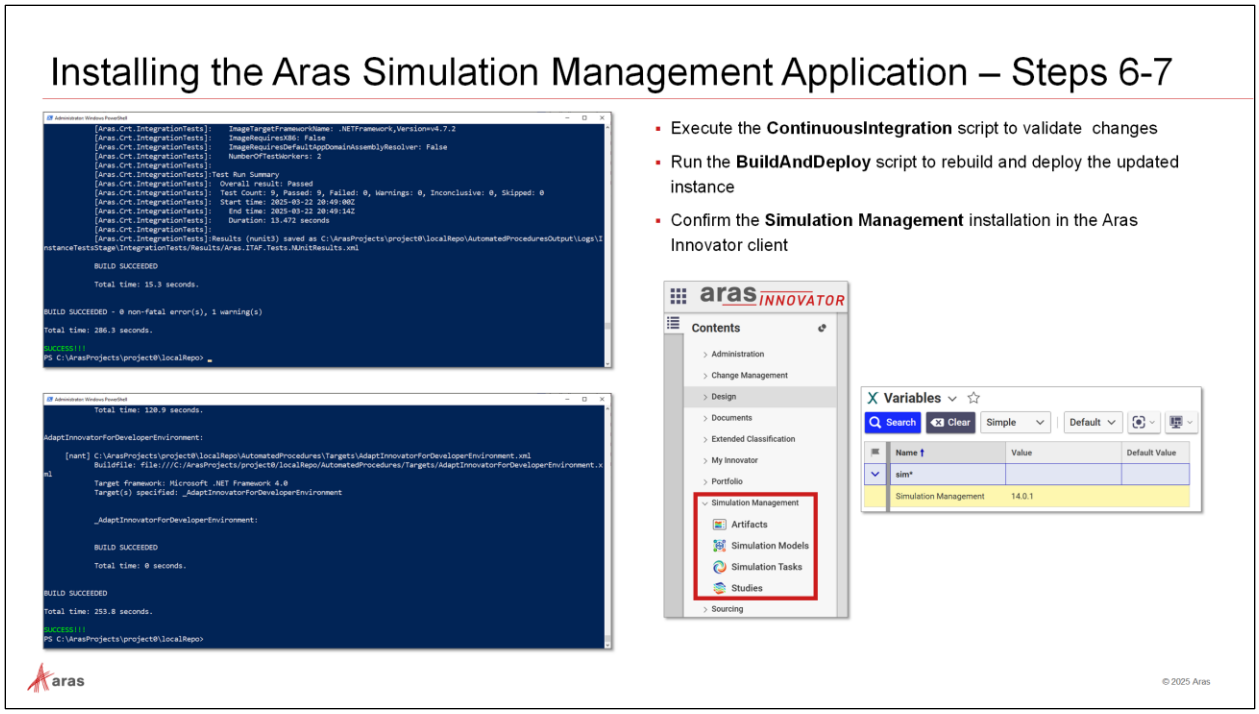
Installing the Aras Simulation Management Application – Step 5

In this lab, you will review, stage, and commit the files that were added or modified during the Simulation Management (SM) application setup.

Lab: Stage and Commit Simulation Management Changes

1. Open Git Extensions for the repository **..\project0\LocalRepo**.
2. In Git Extensions, go to the **Commit to AxxTrngNN** window, and **stage** the following files and folders:
   a. All files in **AML-packages\SM\Import** folder
   b. All folders and files in **CodeTree\Innovator\Client\Solutions\SM** directory
   c. Updated file **imports.mf**
3. In the same window, enter a **descriptive commit message**, such as:
   US-### Added Simulation Management artifacts
4. Click **Commit**, then click **OK** in the **Process** window to confirm the operation.

Reminder: each commit should include **only related changes**. This practice improves collaboration and helps reviewers easily identify updates tied to specific applications or configurations.



Installing the Aras Simulation Management Application – Steps 6-7

In the following labs, you will validate your work by running the **ContinuousIntegration** pipeline, then rebuild and deploy the updated solution using **BuildAndDeploy**. Finally, you can confirm that the Simulation Management (SM) application is properly installed in the Aras Innovator client.

Lab 1: Execute ContinuousIntegration Pipeline to Validate the Build

1. In **Git Extensions**, ensure the **AxxTrngNN** branch is checked out.
2. Open an **Administrator Windows PowerShell** session.
3. Navigate to the local repository:
   C:\ArasProjects\project0\localRepo
4. Run the Continuous Integration script:
   .\ContinuousIntegration.ps1
5. Review the script output:
   a. Confirm **BUILD SUCCEEDED** with zero fatal errors.
   b. If failures occur, address the issues (e.g., missing AML imports, incorrect package paths), then rerun the script.

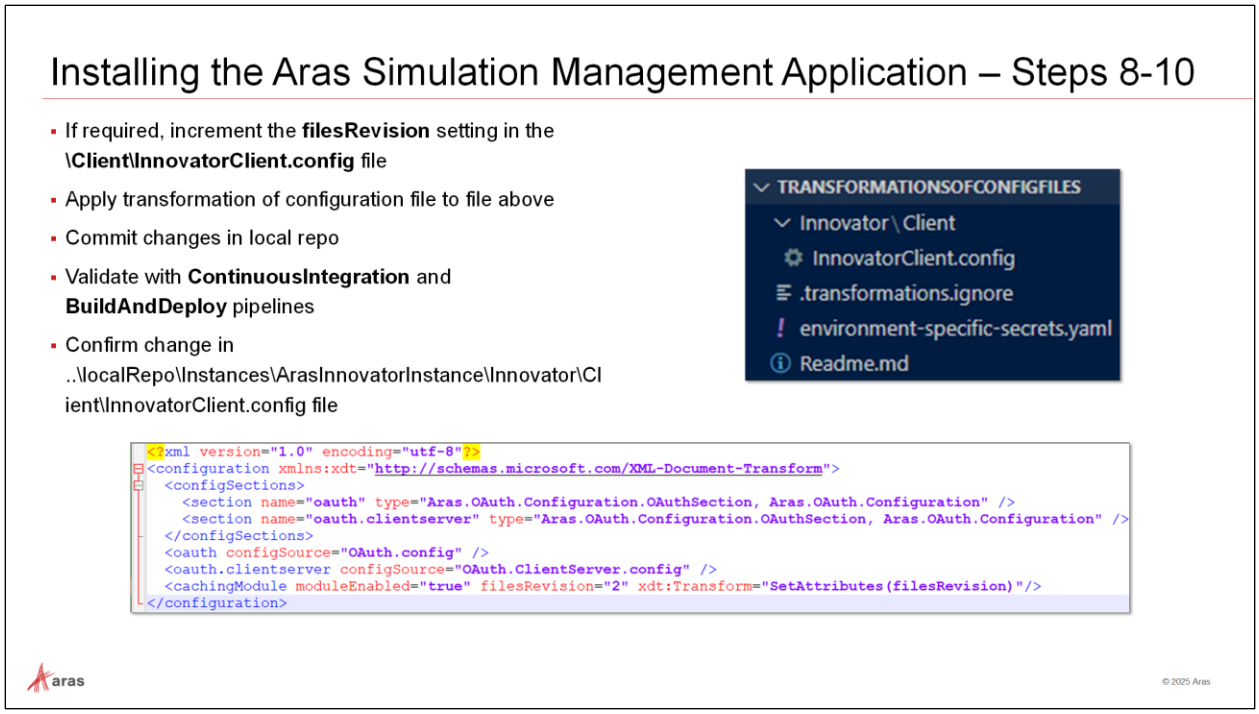
Lab 2: Execute BuildAndDeploy Pipeline to Confirm the Build

1. In **Git Extensions**, ensure the **AxxTrngNN** branch is checked out.
2. Open an **Administrator Windows PowerShell** session.
3. Navigate to the local repository:
   C:\ArasProjects\project0\localRepo
4. Run the Build and Deploy script:
   .\BuildAndDeploy.ps1
5. Confirm deployment results:
   a. A **green "SUCCESS!!!" message** indicates a successful build and deployment.

   b. If the script fails, review the log output and correct issues (common causes include missing AML files or incorrect manifest entries).

Lab 3: Confirm the Simulation Management Installation

1. Log in to the **Aras Innovator client** using administrative credentials.
2. In the TOC, navigate to:
   Administration → Variables
3. Verify that the **Simulation Management** variable appears with the expected version.
4. Confirm that the **Simulation Management** menu items (Artifacts, Simulation Models, Simulation Tasks, Studies, etc.) are now available in the TOC.



Installing the Aras Simulation Management Application – Steps 8-10

In these steps, you will apply any required hotfixes or configuration updates for the Simulation Management (SM) application. This includes updating configuration files through the transformation mechanism, validating the results, and confirming that the changes are reflected in the newly deployed Aras Innovator instance.

Lab 1: Apply Configuration Updates (If Required)

1. In **Git Extensions**, ensure the **AxxTrngNN** branch is checked out.
2. **Check whether the application requires a configuration update**, such as updating the filesRevision value in: **\Client\InnovatorClient.config**

   Note: The updated revision ensures that client-side resources are refreshed.
3. In File Explorer, copy the file **InnovatorClient.config** in folder **..\localRepo\Instances\ArasInnovatorInstance\Client** into the folder **..\localRepo\TransformationsOfConfigFiles\Innovator\Client**
4. Replace contents with the following and save:

```
<?xml version="1.0" encoding="utf-8"?>
<configuration xmlns:xdt="http://schemas.microsoft.com/XML-Document-Transform">
  <configSections>
    <section name="oauth" type="Aras.OAuth.Configuration.OAuthSection,
Aras.OAuth.Configuration" />
    <section name="oauth.clientserver" type="Aras.OAuth.Configuration.OAuthSection,
Aras.OAuth.Configuration" />
  </configSections>
  <oauth configSource="OAuth.config" />
  <oauth.clientserver configSource="OAuth.ClientServer.config" />
  <cachingModule moduleEnabled="true" filesRevision="2"
xdt:Transform="SetAttributes(filesRevision)"/>
</configuration>
```

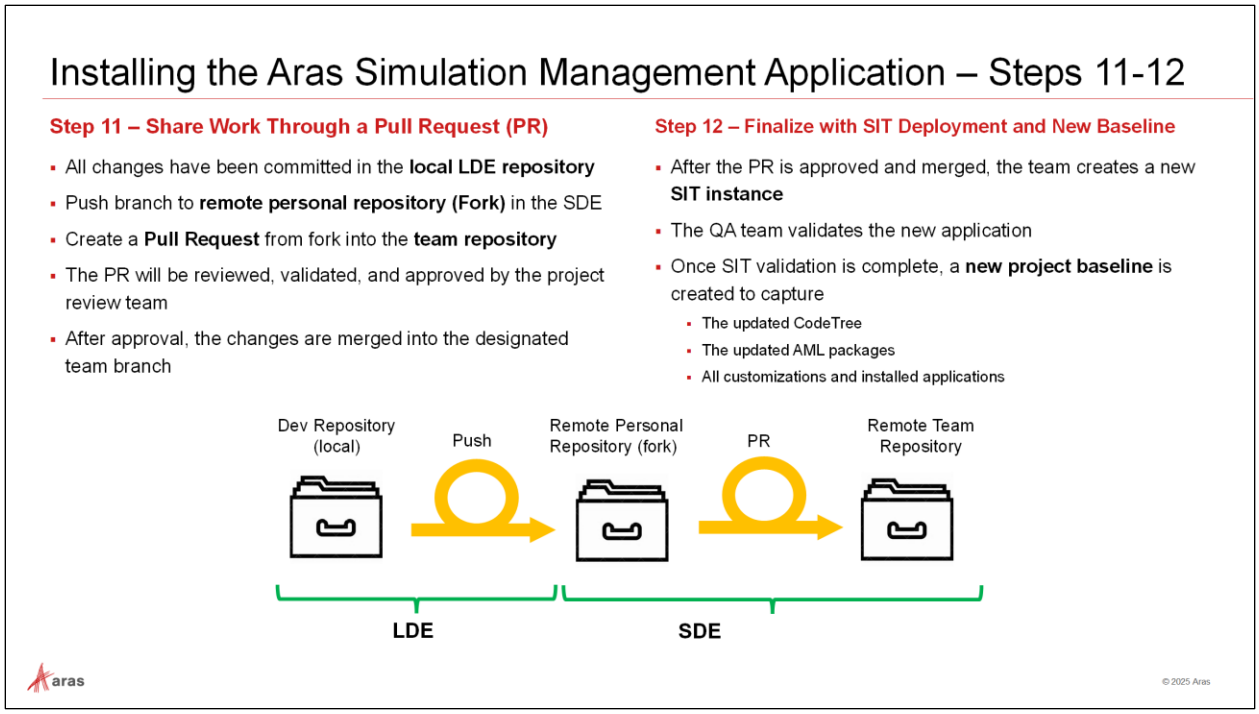
5. In Git Extensions, commit the file above in the **Commit to AxxTrngNN** window, after addition of a proper commit message, e.g.,
   "Transformed InnovatorClient.config file"
6. Click **OK** in **Process** window.

Lab 2: Validate Configuration Updates

1. Run the **ContinuousIntegration** pipeline to validate your changes:
   .\ContinuousIntegration.ps1
   Resolve any transformation or validation errors before proceeding.
2. After a successful CI run, execute:
   .\BuildAndDeploy.ps1
   This ensures the updated configuration is included in the rebuilt Innovator instance.

Lab 3: Confirm Configuration Updates in the Deployed Instance

1. Navigate to the deployed instance folder:
   ..\localRepo\Instances\ArasInnovatorInstance\Innovator\Client
2. Open the new **InnovatorClient.config** file.
3. Confirm that your updates (updated filesRevision value) appear correctly in the deployed file.



Installing the Aras Simulation Management Application – Steps 11-12

At this point, the Simulation Management (SM) application has been installed, configured, and validated in your Local Development Environment (LDE). The remaining steps complete the contribution workflow by pushing the changes to the SDE, initiating a Pull Request (PR), and finalizing the project in the SIT environment.

Notes:

- You may also add the **Administrators** identity to the **Simulation Administrator** identity to ensure the Simulation Management application artifacts are displayed in the TOC.
- You will need to export, stage, and commit the change before you do the labs below.

Lab 1: Push Changes to Fork

1. Navigate to folder **C:\ArasProjects\project0\localRepo**.
2. From the **localRepo** folder context menu, click **GitExt Open repository**.
3. In the left vertical pane, expand the **Branches** node.
4. Check out the **AxxTrngNN** branch (if not already done).
5. In the **Commands** menu, click the **Push** option.
6. In the **Push** window, select the following options:
   a. **Remote**: *xxTeam*
   b. **URL**: *your SDE Fork URL*
   c. **Push branch** tab:
      1. **Branch to push**: *AxxTrngNN*
      2. **To**: *AxxTrngNN*
7. Click **Push**.
8. Click **OK** in the “tracking reference” pop-up window that may appear.
9. Click **OK** in the **Push to xxTeam** dialog.

10. In the left vertical pane, expand the **Remotes** node.
11. In the **xxFork** remote, click the **AxxTrngNN** branch.
12. Verify that the Fork **AxxTrngNN** branch is **now** aligned with your local repo **AxxTrngNN** branch (execute a **Fetch and prune** operation if needed).
13. In a web browser, go to your Azure DevOps SDE fork.
14. In the **AxxTrngNN** branch, confirm that the **AML-packages** folder contains your new package.

Lab 2: Create a New PR and Process to Approval

1. Navigate to: https://dev.azure.com/ArasTraining/ArasTraining-SaaS-Implementation
2. Go to any page in the **Repos** service, and click the **Create a pull request** button, in the system-generated fork update message.
3. Create the new PR with the following details:
   a. **Source Repo/Branch**: your Fork/**AxxTrngNN**.
   b. **Target Repo/Branch**: team repo/**AxxTrngNN**.
   c. **Title** and **Description**:
      1. **Title**: *Axx Pull Request US-xyz*
      2. **Description**: *Pull request for User Story xyz*
4. Click the **Create** button to finalize the creation of the new PR.
5. To enable PR automatic completion, click **Set auto-complete > Set auto-complete**, and select the following in the **Enable automatic completion** window:
   a. **Merge type**: *Rebase and fast-forward*
   b. **Post-completion options**:
      1. **Complete associated work items after merging**: *checked*
      2. **Delete AxxTrngNN after merging**: *unchecked*
   c. Click **Set auto-complete** to close dialog
6. Verify that the message "**Auto-complete: …**" appears in the **Overview** tab for your PR.

Lab 3: Create a Tag on the Last Approved commit

1. Navigate to: https://dev.azure.com/ArasTraining/ArasTraining-SaaS-Implementation
2. Select the **Work** repo (team repo) in **Repos**.
3. Select the branch for which you ran your successful PR workflow and go to **Commits**.
4. From the last successful commit, click the **More options** menu (far right) and select **Create tag**.
5. Enter the following details in the **Create a tag** dialog:
   a. **Name**: *AxxTrngNN* (xx, as provided by instructor)
      For example: **A01Trng1a**.
   b. **Based on**: *commit above*.
   c. **Description**: *New baseline with new Aras applications*.
6. Click **Create**.

Lab 4: Create a New SIT Instance

1. Navigate to https://dev.azure.com/ArasTraining/ArasTraining-SaaS-Implementation.
2. Go to **Pipelines > Pipelines**.
3. Click the **DeployInnovatorAicd** pipeline.
4. Click the **Run pipeline** button in the top right-hand corner of the **DeployInnovatorAicd** page.
5. In the **Run pipeline** window, enter the following details:
   a. **Branch/tag**: *select the Team repo branch from the last approved PR*.
   b. **Deployment type**: *SIT*.
   c. **Other settings**: *default settings*.
   d. **Advanced options**: *default settings (except Resources below)*.
   e. **Enable system diagnostics**: *unchecked*.
6. Click on **Resources** in **Advanced options**:
   a. Click **ContinuousIntegrationAicd [CI]** (last successful run).
   b. Select the last successful CI for the branch above.
   c. Click **Use selected run**.
7. Navigate back to the **Run pipeline** window and click **Run**.
8. The pipeline is queued by the system, and its progress may be viewed from the **Runs** tab of the **DeployInnovatorAicd** page, and by clicking the **Deploy job** link in the pipeline run page.
9. When the pipeline is completed, a link to the new instance should be available on the Wiki page for testing.

Lab 5: Execute the GenerateBaselineAicd Pipeline

1. Navigate to https://dev.azure.com/ArasTraining/ArasTraining-SaaS-Implementation.
2. Go to **Pipelines > Pipelines**.
3. Click the **GenerateBaselineAicd** pipeline.
4. The **GenerateBaselineAicd** pipeline **Runs** page opens, with last executions.
5. Click the **Run pipeline** button in the top right-hand corner of the **GenerateBaselineAicd** page.
6. In the **Run pipeline** dialog, enter the following details:
   a. **Branch/tag**: *4aTrngA00* (tag created in previous lab)
   b. **Advanced options**: *default settings*
   c. **Enable system diagnostics**: *unchecked*
   d. **Pipeline timeout in minutes**: *0*
7. **Click Run**.
8. The pipeline is queued by the system, and its progress may be viewed from the **Runs** tab of the **GenerateBaselineAicd** page, and by clicking the **Stages** and **Jobs** tabs in the pipeline run page.
9. The new baseline will be uploaded to the storage account in the artifacts feed, i.e., **Artifacts > SaaS-Baselines**.

Summary

In this unit, you learned how to **install and manage Aras Innovator applications** within the **Aras DevOps framework**. You also learned how to **prepare both the Standard Development Environment (SDE)** and the **Local Development Environment (LDE)** to support new Aras Innovator applications.

After completing the setup and installation, you **pushed applications from the LDE to the SDE** and finally **deployed the applications** in a new **System Integration Testing (SIT)** instance.

You should now be able to:

- **Prepare the SDE** for new development work
- **Identify and verify environment prerequisites** for the LDE
- **Set up the LDE** to support new Aras Innovator applications
- **Install and configure Aras Innovator applications** in the LDE
- **Validate installations** to ensure successful deployment in the LDE
- **Share and synchronize applications** within the SDE
- **Validate Aras Innovator applications** in the SDE environment
- **Finalize the project** by establishing a new baseline for future development

For more information, please go to the following Aras web sites:

- https://docs.aras.com/
- https://www.aras.com/community/DocumentationLibrary/Library/LibraryStart.htm
- https://www.aras.com/community/subscriber-portal/training/w/development-best-practices/1003/aras-devops-training
- https://www.aras.com/community/DocumentationLibrary/Library/Aras%20DevOps.htm
- https://www.aras.com/community/DocumentationLibrary/Library/Aras%20Enterprise.htm
- https://www.aras.com/community/subscriber-portal/training/w/saas-devops
- https://aras.com/en/why-aras/plm-software-as-a-service